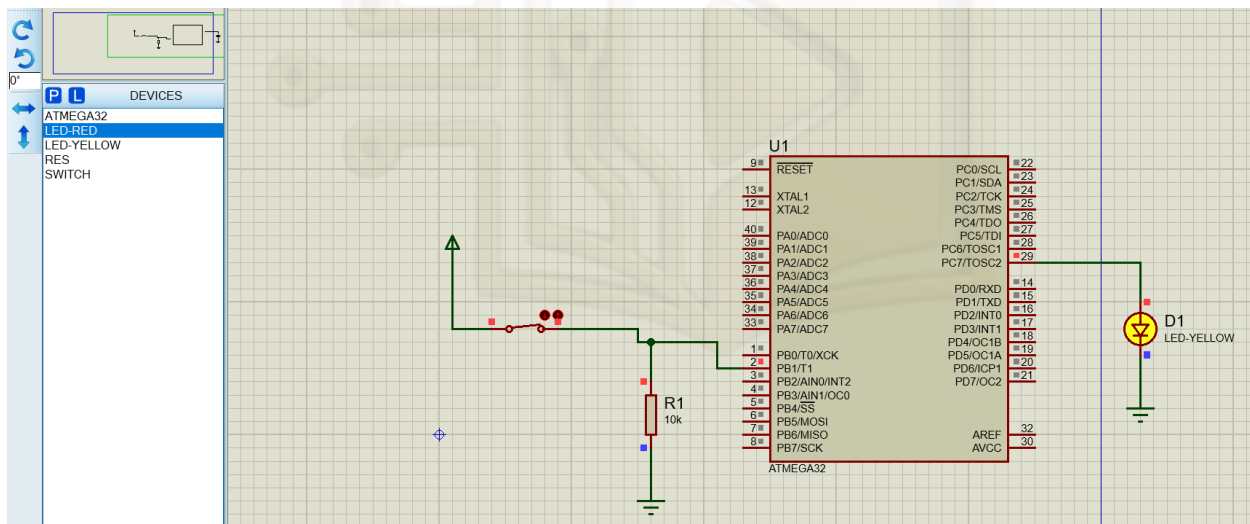


1. A door sensor is connected to bit 1 of port B and LED is connected to bit 7 of port C.
Monitor the door sensor, and when it opens, turn the light

Program

```
#include <avr/io.h> // standard AVR header
int main(void)
{
    DDRB = DDRB & 0b11111101; // pin 1 of Port B is input
    DDRC = DDRC | 0b10000000; // pin 7 of Port C is output
    while (1)
    {
        if (PINB & 0b00000010) // check pin 1 (2nd pin) of PINB
            PORTC = PORTC | 0b10000000; // set pin 7 (8th pin) of PORTC
        else
            PORTC = PORTC & 0b01111111; // clear pin 7 (8th pin) of PORTC
    }
    return 0;
}
```

Circuit Configuration



2. 7-Segment Display

Program

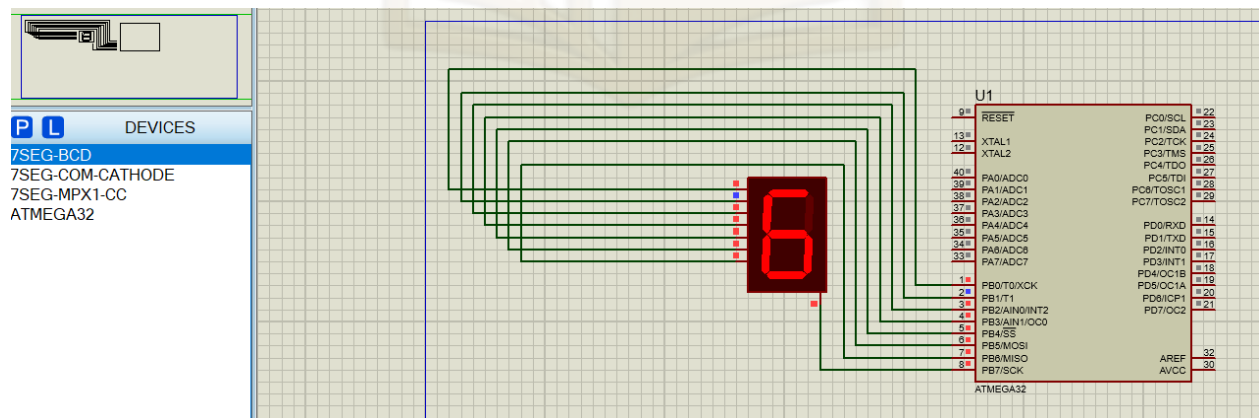
```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
unsigned char seg_code[10] = {
    0x3F, // 0
    0x06, // 1
```

```

0x5B, // 2
0x4F, // 3
0x66, // 4
0x6D, // 5
0x7D, // 6
0x07, // 7
0x7F, // 8
0x6F // 9
};
int main(void)
{
    DDRB = 0xFF; // PORTB as output
    while(1)
    {
        for(int i = 0; i < 10; i++)
        {
            PORTB = seg_code[i];
            _delay_ms(100);
        }
    }
}

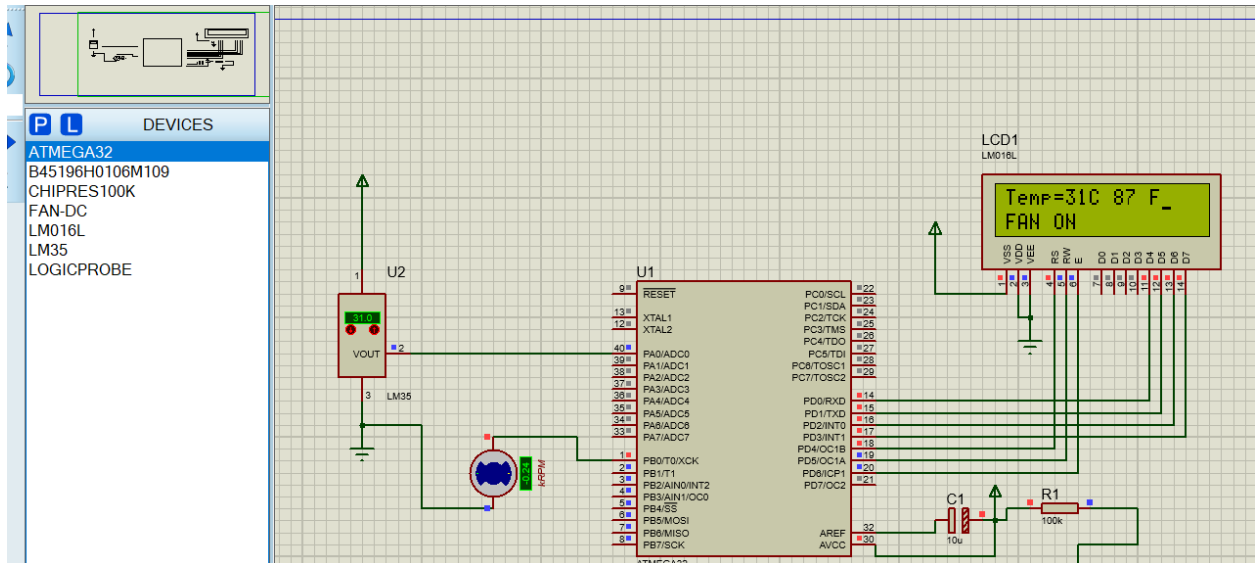
```

Circuit Configuration



3. Temperature Sensors

Use the folder



4. LCD with ABCD display Program

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

/* LCD Control Pins */
#define LCD_CTRL PORTC
#define LCD_CTRL_DDR DDRC
#define RS PC0
#define RW PC1
#define EN PC2

/* LCD Data Port */
#define LCD_DATA PORTB
#define LCD_DATA_DDR DDRB

void lcd_cmd(unsigned char cmd)
{
    LCD_DATA = cmd;
    LCD_CTRL &= ~(1 << RS); // RS = 0
    LCD_CTRL &= ~(1 << RW); // RW = 0
    LCD_CTRL |= (1 << EN);
    _delay_ms(2);
    LCD_CTRL &= ~(1 << EN);
}

void lcd_data(unsigned char data)
```

```

{
    LCD_DATA = data;
    LCD_CTRL |= (1 << RS); // RS = 1
    LCD_CTRL &= ~(1 << RW); // RW = 0
    LCD_CTRL |= (1 << EN);
    _delay_ms(2);
    LCD_CTRL &= ~(1 << EN);
}

void lcd_init()
{
    LCD_DATA_DDR = 0xFF;
    LCD_CTRL_DDR |= (1 << RS) | (1 << RW) | (1 << EN);

    _delay_ms(20);
    lcd_cmd(0x38); // 8-bit, 2 line
    lcd_cmd(0x0C); // Display ON
    lcd_cmd(0x06); // Auto increment
    lcd_cmd(0x01); // Clear display
    _delay_ms(2);
}

int main(void)
{
    lcd_init();

    lcd_data('A');
    _delay_ms(100);

    lcd_data('B');
    _delay_ms(100);

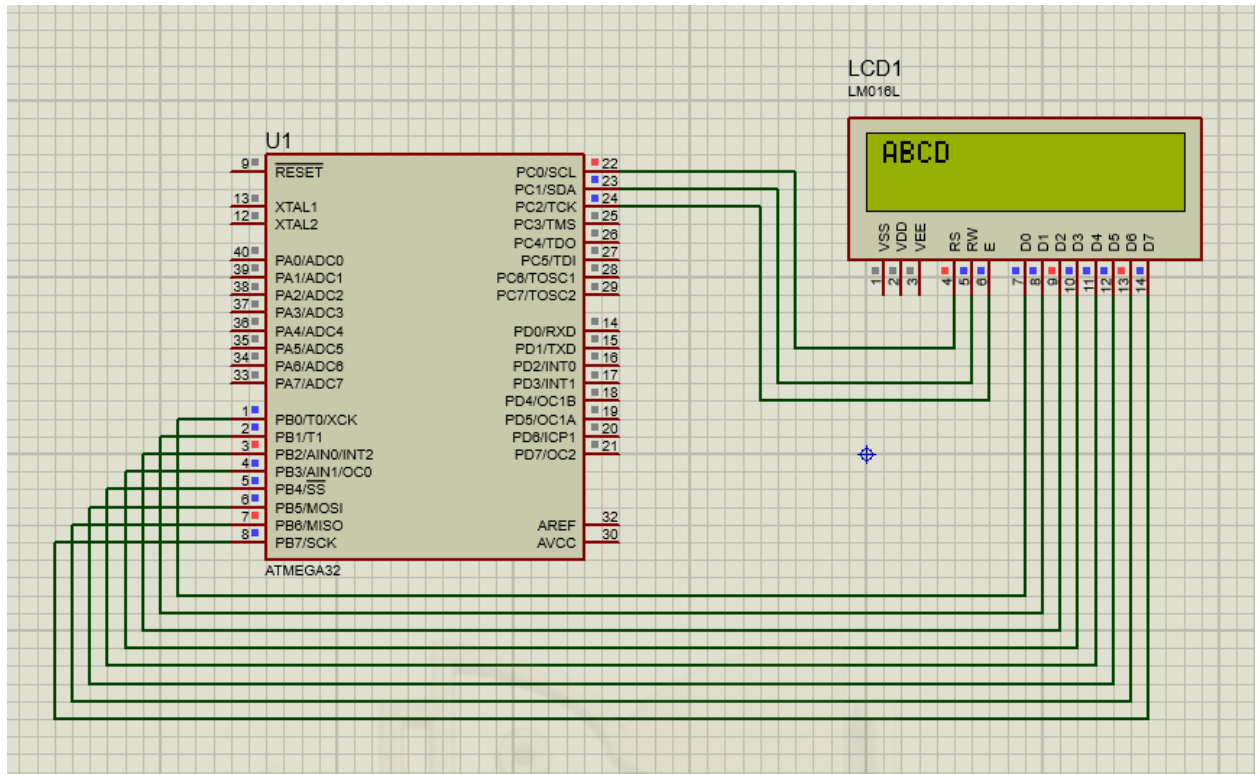
    lcd_data('C');
    _delay_ms(100);

    lcd_data('D');
    _delay_ms(100);

    while (1);
}

```

Circuit Diagram



5. Your Name on LCD Program

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

/* LCD Control Pins */
#define LCD_CTRL PORTC
#define LCD_CTRL_DDR DDRC
#define RS PC0
#define RW PC1
#define EN PC2

/* LCD Data Port */
#define LCD_DATA PORTB
#define LCD_DATA_DDR DDRB

void lcd_cmd(unsigned char cmd)
{
    LCD_DATA = cmd;
    LCD_CTRL &= ~(1 << RS); // RS = 0
    LCD_CTRL &= ~(1 << RW); // RW = 0
    LCD_CTRL |= (1 << EN);
```

```

    _delay_ms(2);
    LCD_CTRL &= ~(1 << EN);
}

void lcd_data(unsigned char data)
{
    LCD_DATA = data;
    LCD_CTRL |= (1 << RS); // RS = 1
    LCD_CTRL &= ~(1 << RW); // RW = 0
    LCD_CTRL |= (1 << EN);
    _delay_ms(2);
    LCD_CTRL &= ~(1 << EN);
}

void lcd_string(char *str)
{
    while (*str)
    {
        lcd_data(*str);
        str++;
    }
}

void lcd_init()
{
    LCD_DATA_DDR = 0xFF;
    LCD_CTRL_DDR |= (1 << RS) | (1 << RW) | (1 << EN);

    _delay_ms(20);
    lcd_cmd(0x38); // 8-bit, 2 line, 5x7
    lcd_cmd(0x0C); // Display ON, cursor OFF
    lcd_cmd(0x06); // Increment cursor
    lcd_cmd(0x01); // Clear display
    _delay_ms(2);
}

int main(void)
{
    lcd_init();

    lcd_cmd(0x80); // First line
    lcd_string("SIMANTA");

    lcd_cmd(0xC0); // Second line

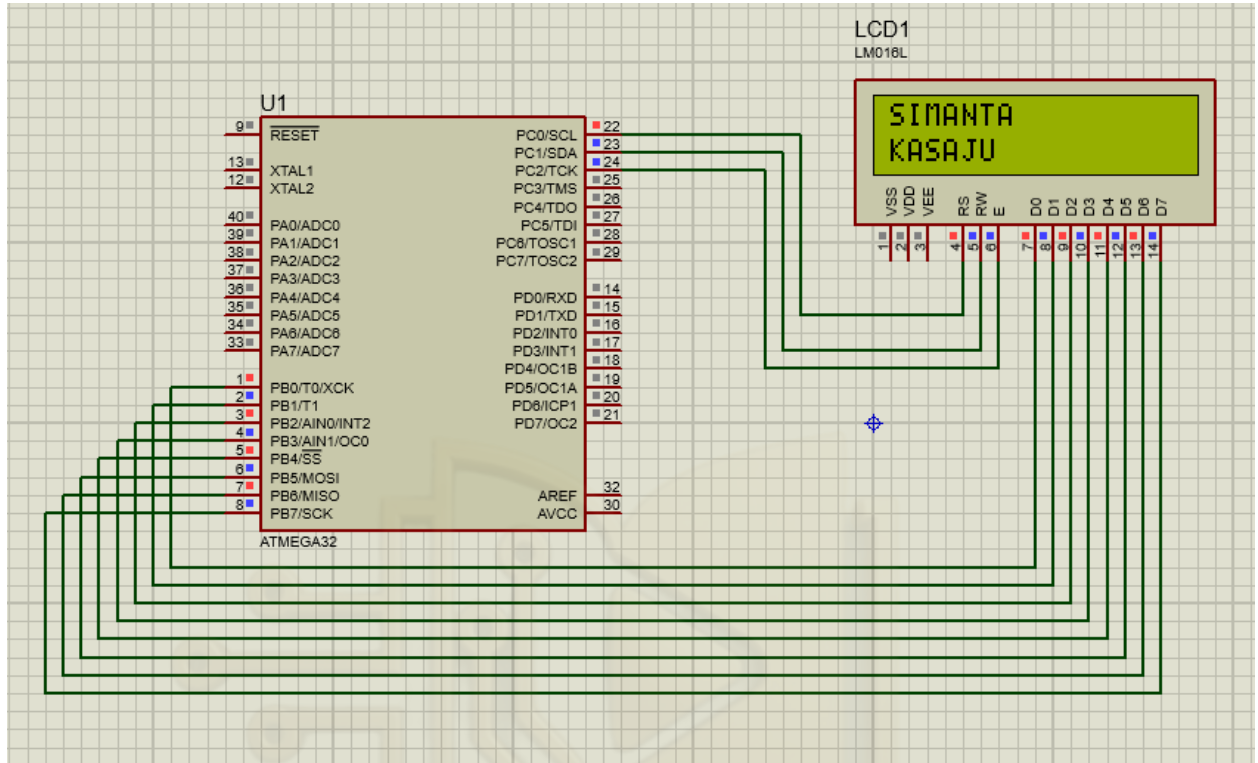
```

```

lcd_string("KASAJU");

while (1);
}

```



6. Counter Program

```

#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h> // for itoa

/* LCD Control Pins */
#define LCD_CTRL PORTC
#define LCD_CTRL_DDR DDRC
#define RS PC0
#define RW PC1
#define EN PC2

/* LCD Data Pins */
#define LCD_DATA PORTB
#define LCD_DATA_DDR DDRB

```

```

/* LCD Functions */
void lcd_cmd(unsigned char cmd)
{
    LCD_DATA = cmd;
    LCD_CTRL &= ~(1 << RS);
    LCD_CTRL &= ~(1 << RW);
    LCD_CTRL |= (1 << EN);
    _delay_ms(2);
    LCD_CTRL &= ~(1 << EN);
}

void lcd_data(unsigned char data)
{
    LCD_DATA = data;
    LCD_CTRL |= (1 << RS);
    LCD_CTRL &= ~(1 << RW);
    LCD_CTRL |= (1 << EN);
    _delay_ms(2);
    LCD_CTRL &= ~(1 << EN);
}

void lcd_string(char *str)
{
    while (*str)
        lcd_data(*str++);
}

void lcd_init()
{
    LCD_DATA_DDR = 0xFF;
    LCD_CTRL_DDR |= (1 << RS) | (1 << RW) | (1 << EN);

    _delay_ms(20);
    lcd_cmd(0x38); // 8-bit, 2 line
    lcd_cmd(0x0C); // Display ON
    lcd_cmd(0x06); // Auto increment
    lcd_cmd(0x01); // Clear LCD
}

/* Timer1 1-second delay */
void timer1_delay_1s()
{
    TCNT1 = 49911; // preload for 1 second
    TCCR1A = 0x00;

```

```

TCCR1B = (1 << CS12) | (1 << CS10); // Prescaler 1024

while (!(TIFR & (1 << TOV1))); // Wait for overflow
TIFR |= (1 << TOV1); // Clear overflow flag
}

int main(void)
{
    char buffer[6];
    int count = 0;

    lcd_init();
    lcd_cmd(0x80);
    lcd_string("Counter:");

    while (1)
    {
        lcd_cmd(0xC0); // Second line
        itoa(count, buffer, 10); // int to string
        lcd_string(buffer);

        count++;
        timer1_delay_1s();
        lcd_cmd(0x01); // Clear LCD
        lcd_cmd(0x80);
        lcd_string("Counter:");
    }
}

```

Circuit

